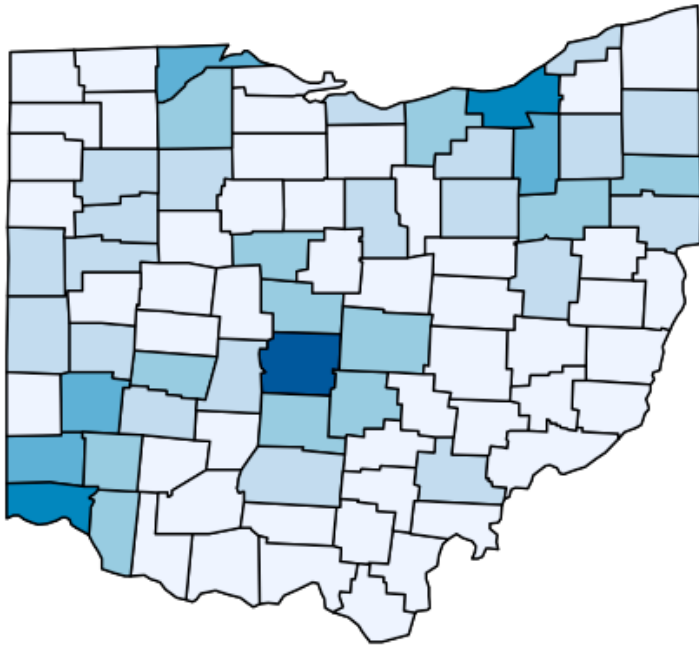


State of Ohio | COVID – 19 Dashboard



Cases
200231



Hospitalizations
18235



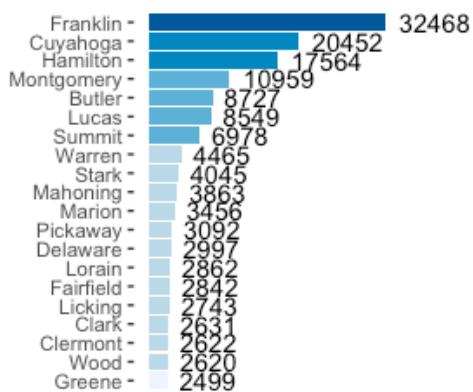
Deaths
5217



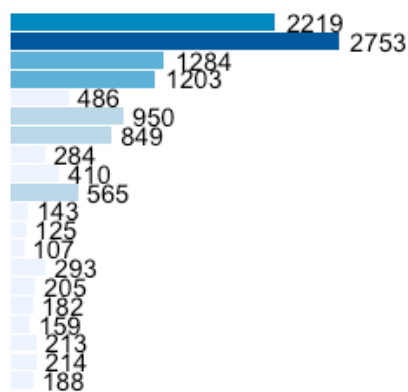
Presumed Recovered
181476



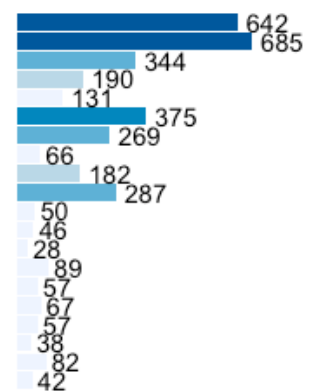
Case Count



Hospitalization Count



Death Count



Process

One of the hardest things about this project was the planning and set up before the actual code and implementation. As a computer scientist, I was mulling over the best possible method to create the data sets, graphs, and how to do it in the most clear and concise way possible. I spent a while rummaging through class and homework code, and also just looking at the Ohio Dashboard, coming up with a plan for my display. Before I get to the long story, the short story is that I used many, many examples from our past coding experiences to shape my data, and to bring my graphs to life, of course with a little help from the internet (mainly R Graphics Cookbook) as well.

I came to the conclusion that I wanted to build all my graphs first before any sort of arrangement, coloring, or cleaning up into a dashboard. I went to the map first since we had just done that for module seven homework, so I reused a lot of the code to produce my image of Ohio with the counties drawn correctly. I then brought in the data from the Ohio Dashboard and searched for data regarding population by county for Ohio. Once I had a suitable source for that, I loaded it into a data frame. Then I did some data handling with each of my two data frames to reduce them down to the data I would need for the rest of the project. I combined these two data sets into a new data set so that I had the case counts from the dashboard data matched with the total county populations. I then did a merge with my map data frame so that I could correspond the cases with its respective county. I added the fill variable, but at this point I just had it fill by cases per county in a continuous fashion, as I did not have breaks yet for distinct coloring ranges. I saved my graph to an object because I knew I would need it for later use.

I knew the set of four graphs on the right and the set of three graphs on the bottom would be resulted from making one for each group, and then repasting code for each of the graphs. I'd have to change a variable or two, so I tried to create just one graph at once and then flip the variable for the other graphs later. I began with the three panels on the bottom because on Monday the 26th, I believe we worked with this sort of graph where the bars were horizontal, so it was fresh in my mind. I remembered that you said we did not need to display all 88 counties for this so I looked at only the top 20, ordered by number of cases for each county for each of the bottom three graphs. I brought in a lot of our previous code to get my graph started. Then I went to work on eliminating the axis titles, lines, and ticks, just as it appears on the Ohio Dashboard, which I used R Graphics Cookbook for. I also used it to set an x-axis limit greater than what the bars stretched too since I was going to add the number of cases for each county at the end of their bar. That took a simple `geom_text` which I remembered using in a previous homework so I resourced to help create this one. Once again, I let the shading go, and moved onto the next graph as I had my main basics down for what I wanted the bottom graphs to look like.

For the side panel of graphs, I had to edit a previous data frame so that I could changing the group to date instead of by county since these graphs are measured along the scale of time. I then went to work on the graph, beginning it from scratch and then incorporating code from the previous graph I had just made to once again clean up the axes. This graph did not take me quite as long, and got my base done pretty quickly. I did have to create another data set for the depiction of the presumed recovered patients. I brought into today's date and filtered for cases that occurred over 10 days ago since those would be my presumed recovered cases. I also dropped the count of dead cases from my count of total cases since those are not recoveries. Then creating the graph was extremely similar except I did add 10 to my x variable of onset date since this was now recovery date.

Now all my base graphs were done, so I went back to bring them up to Ohio Dashboard level. I went back to my beginning OhioCountyDF and added categorical variables that had breaks according to those matching on the dashboard (or at least as close as I get). I went back into my Ohio map graph and changed the fill to categorical variable for cases by county so that I would get the shading that is currently shown on my dashboard. I then dropped down to my bottom graph, and added the categorical variable for that fill as well. I then copied this code over for the other two graphs, and changed the couple variables needed so that it would show either Deaths or Hospitalizations instead of Cases. Also went through and changed the title of each graph and customized them so that they did not have the left y-axis or ticks anymore since that was only for the first graph. I then headed over to my right panel of graphs, copied my code for cases into my other two graphs for Deaths and Hospitalizations again, and changed the appropriate variables and labels. I then noticed that the Ohio Dashboard had the totals for each of the right side panel graphs in big red lettering under the title, so I went back and added that for each of those four graphs, using some code that we had used previously in this class. Once I had my bottom panel of graphs done, and my left panel of graphs done, I combined them into respective graph objects, so that the similar graphs would all be under one object. So now I was working with three objects: my map, my right panel, and my bottom panel.

Combining them into one singular graph may have been the most difficult part. I originally tried to use patchwork, but the tools I was using didn't seem to have much customization to them, so I changed to using pushViewport. We had done this sort of grid graphing before so I referenced that to get my graphs into their correct locations. Except now the right panel graphs were smashed into each other and the actual data was missing, in the same way that they were for patchwork. However, now I had a customization tool I knew about so I messed with the row and column numbers for a while, trying to expand them out but just could not get the right panel to expand. Only after I came to office hours did I realize that since I kept scaling up the state of Ohio map with these 4 side panels, the side panels would stay their same respect size since they had to fit within the height of my Ohio graph. I then reduced my Ohio graph size, and kept my right panel graphs the same size as they were, and this allowed them to expand and show the data. It took a lot trial and error to find precisely what I liked best, but I'm happy with what it looks like now.

Overall, I'd like to think I stayed pretty organized when creating the project. I got the basics down, and then worked on the nitty-gritty and fine tuning after, polishing it up to my final solution. The only real trouble I had was just getting the data to display on the dashboard which I used office hours for to get that fixed. Otherwise, the other methods I used were for the most part in previous code we had used, or a quick google search away.

References

R: R Core Team (2020). R: A language and environment for statistical computing. R Foundation for Statistical Computing, Vienna, Austria.
<https://www.R-project.org/>.

Libraries:

tidyverse: Wickham et al., (2019). Welcome to the tidyverse. Journal of

Open Source Software, 4(43), 1686,
<https://doi.org/10.21105/joss.01686>

ggthemes: Jeffrey B. Arnold (2019). ggthemes: Extra Themes, Scales and
Geoms for 'ggplot2'. R package version 4.2.0.
<https://CRAN.R-project.org/package=ggthemes>

tidyquant: Matt Dancho and Davis Vaughan (2020). tidyquant: Tidy
Quantitative Financial Analysis. R package version 1.0.1.
<https://CRAN.R-project.org/package=tidyquant>

patchwork: Thomas Lin Pedersen (2020). patchwork: The Composer of
Plots. R package version 1.0.1.
<https://CRAN.R-project.org/package=patchwork>

scales: Hadley Wickham and Dana Seidel (2020). scales: Scale
Functions for Visualization. R package version 1.1.1.
<https://CRAN.R-project.org/package=scales>

grid: R Core Team (2020). R: A language and environment for
statistical computing. R Foundation for Statistical Computing,
Vienna, Austria. <https://www.R-project.org/>.

Other Resources:

Ohio COVID data: <https://coronavirus.ohio.gov/wps/portal/gov/covid-19/dashboards>

Ohio county populations: https://www.census.gov/data/datasets/time-series/demo/popest/2010s-counties-total.html#par_textimage_739801612

Rename function: <https://www.sharpsightlabs.com/blog/rename-columns-in-r/>

Top 20 Counties: <https://stackoverflow.com/questions/57175180/how-can-i-show-only-the-top-10-categories-and-values-in-geom-point-dot-plot>

Many various bit: <https://r-graphics.org/>

Code

```
library(tidyverse)
library(ggthemes)
library(tidyquant)
library(patchwork)
library(scales)
library(grid)
```



```

#filter out unneeded rows and columns from total ohio county
#populations before combining tables
ohioPop <- ohioPop[5:92,2]

#match total ohio county populations with covid statistics per
#county so that we can analyze rates of infection.
#rename county to region to map it with ohio.county
#https://www.sharpsightlabs.com/blog/rename-columns-in-r/
combinedOhio <- cbind(ohioCountyDF, ohioPop) %>%
  mutate(countyPop = as.double(`X2`)) %>%
  select(!`X2`) %>%
  mutate(rate = (ncases/countyPop) * 1000) %>%
  mutate(County = str_to_lower(County)) %>%
  rename(subregion = County)

#combine Ohio state map and previously binded data set together
#so that data can be displayed on Ohio map by county
rateDB <- merge(ohio.county, combinedOhio, by.x = "subregion",
               by.y = "subregion")

#Ohio map with discrete shading by total cases
ohioMap <- ggplot(data = rateDB, aes(x=long,y=lat,group = group,
                                     fill=CaseCat)) +
  geom_polygon(color = "black") +
  scale_fill_brewer() +
  guides(fill = FALSE) +
  theme_map()

# Bottom panels -----

#process for bottom 3 graphs written here since the same process
#was repeated for each:
#filtered and ordered only the top 20 counties by cases for
#each graph, colored categorically by case amount, with most of
#the aesthetics removed for the graphs to match the cleaned up
#look on the real ohio dashboard. Labeled total for whatever
#category (cases, hospitalizations, deaths) at the end of the
#bar instead of axis labels

#Bottom left case graph which displays the top 20 counties by
#cases

```

```

#used to figure out how to retrieve top 20
#https://stackoverflow.com/questions/57175180/how-can-i-show-
only-the-top-10-categories-and-values-in-geom-point-dot-plot
Cases <- ggplot(data = ohioCountyDF[tail(order(
  ohioCountyDF$ncases), 20), ],
  aes(y=reorder(County, ncases), x=ncases, fill =
    CaseCat)) +
  geom_col() +
  scale_fill_brewer() +
  guides(fill = FALSE) +
  geom_text(aes(label = ncases), hjust = -.2) +
  theme(axis.text.x = element_blank(), axis.ticks.x =
    element_blank(), plot.title = element_text(hjust = 0.5),
    panel.background = element_blank(), panel.grid.major =
    element_blank(), panel.grid.minor = element_blank()) +
  xlim(0, max(ohioCountyDF$ncases + 11000)) +
  labs(y = "", x = "", title = "Case Count")

```

```

#Bottom middle hospitalization graph which displays the top 20
#counties by cases
Hosp <- ggplot(data = ohioCountyDF[tail(order(
  ohioCountyDF$ncases), 20), ],
  aes(y=reorder(County, ncases), x=nhos, fill =
    HosCat)) +
  geom_col() +
  scale_fill_brewer() +
  guides(fill = FALSE) +
  geom_text(aes(label = nhos), hjust = -.2) +
  theme(axis.text = element_blank(), axis.ticks =
    element_blank(), plot.title = element_text(hjust = 0.5),
    panel.background = element_blank(), panel.grid.major =
    element_blank(), panel.grid.minor = element_blank()) +
  xlim(0, max(ohioCountyDF$nhos + 800)) +
  labs(y = "", x = "", title = "Hospitalization Count")

```

```

#Bottom right death graph which displays the top 20 counties by
#cases
Deaths <- ggplot(data = ohioCountyDF[tail(order(
  ohioCountyDF$ncases), 20), ],
  aes(y=reorder(County, ncases), x=ndead, fill =
    DeathCat)) +
  geom_col() +
  scale_fill_brewer() +
  guides(fill = FALSE) +
  geom_text(aes(label = ndead), hjust = -.2) +
  theme(axis.text = element_blank(), axis.ticks =

```

```

        element_blank(), plot.title = element_text(hjust = 0.5),
        panel.background = element_blank(), panel.grid.major =
        element_blank(), panel.grid.minor = element_blank()) +
xlim(0, max(ohioCountyDF$ndead + 550)) +
labs(y = "", x = "", title = "Death Count")

#combine all three graphs into one object
bottom <- Cases + Hosp + Deaths

# Side Panels -----

#Dataset for right panel of four charts, grouped by date of
#infection, created vars for number of cases, hospitalizations
#and for deaths, removed NA values
ohioCasesDF <- ohioDF %>%
  group_by(OnsetDate) %>%
  summarize(ncases = sum(`Case Count`),
            ndead = sum(`Death Due to Illness Count`, na.rm =
                        TRUE),
            nhos = sum(`Hospitalized Count`, na.rm = TRUE))

#breaks for dates on x-axis graph every 2 months
datebreaks <- seq(as.Date("2020-01-01"), as.Date("2020-11-01"),
                  by = "2 months")

#Explanation for 4 right graphs since they are extremely
#similar:
#used my new data set created just above, with my changing
#variable in cases, hospitalizations, deaths or recoveries, but
#always displayed by date of onset. Colored the bars blue, added
#my month labels, cleaned out the axes and labels to reflect
#real dashboard, added subtitle total for each category as is
#with Ohio's dashboard.

#right top case graph
casesBar <- ggplot(data = ohioCasesDF, aes(x = OnsetDate, y =
            ncases)) +
  geom_col(color = "blue") +
  scale_x_date(breaks = datebreaks, labels = date_format("%b
            %d")) +
  scale_fill_brewer() +
  theme(axis.text.y = element_blank(), axis.ticks.y =
        element_blank(), plot.title = element_text(hjust = 0.5),
        panel.background = element_blank(), panel.grid.major =

```



```

        element_blank(), panel.grid.minor = element_blank(),
        plot.subtitle = element_text(hjust = 0.5, size = 20,
        color = "red")) +
labs(y = "", x = "", title = "Cases", subtitle =
paste(sum(ohioCasesDF$ncases)))

#right upper middle hospitalization graph
hosBar <- ggplot(data = ohioCasesDF, aes(x = OnsetDate, y =
        nhos)) +
geom_col(color = "blue") +
scale_x_date(breaks = datebreaks, labels = date_format("%b
        %d")) +
scale_fill_brewer() +
theme(axis.text.y = element_blank(), axis.ticks.y =
        element_blank(), plot.title = element_text(hjust = 0.5),
        panel.background = element_blank(), panel.grid.major =
        element_blank(), panel.grid.minor = element_blank(),
        plot.subtitle = element_text(hjust = 0.5, size = 20,
        color = "red")) +
labs(y = "", x = "", title = "Hospitalizations", subtitle =
paste(sum(ohioCasesDF$nhos)))

#right lower middle death graph
deadBar <- ggplot(data = ohioCasesDF, aes(x = OnsetDate, y =
        ndead)) +
geom_col(color = "blue") +
scale_x_date(breaks = datebreaks, labels = date_format("%b
        %d")) +
scale_fill_brewer() +
theme(axis.text.y = element_blank(), axis.ticks.y =
        element_blank(), plot.title = element_text(hjust = 0.5),
        panel.background = element_blank(), panel.grid.major =
        element_blank(), panel.grid.minor = element_blank(),
        plot.subtitle = element_text(hjust = 0.5, size = 20,
        color = "red")) +
labs(y = "", x = "", title = "Deaths", subtitle =
paste(sum(ohioCasesDF$ndead)))

#Get today's date
Today <- Sys.Date()

#Created for recovery graph, needed to filter out for cases that
#happened more than 10 days ago, and also filter out dead cases
#from total cases

```

```

ohioRec <- ohioCasesDF %>%
  filter(OnsetDate < Today - 10) %>%
  mutate(recovered = ncases - ndead)

#For this graph I had to add 10 to onset date since presumed
#recovered was established as 10 days after onset date, so this
#would effectively show when their 10 days had passed
#right bottom graph for presumed recoveries
recBar <- ggplot(data = ohioRec, aes(x = OnsetDate + 10, y =
  recovered)) +
  geom_col(color = "blue") +
  scale_x_date(breaks = datebreaks, labels = date_format("%b
    %d")) +
  scale_fill_brewer() +
  theme(axis.text.y = element_blank(), axis.ticks.y =
    element_blank(), plot.title = element_text(hjust = 0.5),
    panel.background = element_blank(), panel.grid.major =
    element_blank(), panel.grid.minor = element_blank(),
    plot.subtitle = element_text(hjust = 0.5, size = 20,
    color = "red")) +
  labs(y = "", x = "", title = "Presumed Recovered", subtitle =
    paste(sum(ohioRec$recovered)))

#combined all four right plots into one
right <- casesBar / hosBar / deadBar / recBar

# Pulling it all together -----

#display graphs where they are according to Ohio dashboard, Ohio
#map upper left center, 4 graphs measuring categories by onset
#date on the upper right, and the three graphs measuring totals
#by county on the bottom
pushViewport(viewport(layout = grid.layout(12,6)))
print(ohioMap, vp = viewport(layout.pos.row = 2:7,
  layout.pos.col = 1:3))
print(right, vp = viewport(layout.pos.row = 1:8,
  layout.pos.col = 4:6))
print(bottom, vp = viewport(layout.pos.row = 9:12,
  layout.pos.col = 1:6))

```